

MLIR NPU Lecture 09 Worksheet

목표: MatMul+Bias+ReLU 를 NPU runtime ABI 까지 낮추기 위한 pass pipeline 초안을 작성한다.

Part A - Pass Pipeline Annotation

```
builtin.module(  
  canonicalize,cse,  
  func.func(npu-decompose-composites,npu-fuse-matmul-epilogue),  
  npu-infer-layout,  
  npu-tile-linalg{tile-m=64 tile-n=128 tile-k=32},  
  one-shot-bufferize,  
  npu-assign-memory-space{sram-kb=512},  
  npu-insert-dma{double-buffer=true},  
  npu-place-barriers,  
  npu-emit-runtime-abi{abi-version=1}  
)
```

Pass	Anchor	Input dialect	Output dialect	Why here?
------	--------	---------------	----------------	-----------

Part B - Pass Skeleton Checklist

Item	Answer	Notes
Pass argument name		
Operation anchor		
Dependent dialects		
Options		

Statistics

Required analyses

Failure conditions

Preserved analyses

Part C - NPU Lowering Pipeline Draft

아래 표를 채우세요. Pass 는 최소 8 개 이상 작성합니다.

#	Pass	Anchor	Input -> Output	Analysis	Failure condition	FileCheck strategy
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						

Part D - Reflection

1. Graph fusion 을 bufferization 이후가 아니라 이전에 하는 이유는 무엇인가?
2. Tile size pass 가 필요로 하는 target information 은 무엇인가?

3. DMA/barrier scheduling pass 의 negative test 는 어떤 형태가 되어야 하는가?

4. Pass statistics 로 성능 regression 을 잡으려면 어떤 counter 가 필요한가?